Claude Code: 架构与实现

深入解析 Anthropic 的 AI 编程助手

自动生成的技术文档

2026 年 4 月 1 日

目录

第一章　应用程序引导与入口点

一个健壮的命令行应用程序的启动序列很少是一个简单的从上到下的脚本。对于 Claude Code 来说，引导（bootstrapping）是一个经过精心编排的过程，它在瞬时反馈的需求与初始化丰富运行时环境的复杂性之间取得了平衡。本章将探讨 Claude Code 如何管理其入口点、路由用户命令、协商网络传输，并建立所有后续操作所需的全局状态。

1.1　入口点与 SDK 接口

`entrypoints/` 目录充当 Claude Code 的公共 API 层面。传统的 CLI 通常假设总是有人类用户坐在终端前，而 Claude Code 的设计旨在在多种上下文中运行：作为交互式终端应用程序、作为无头模型上下文协议（MCP）服务器，以及作为外部 SDK（如 Python 和 Rust）的基础引擎。为了在不增加任何单一用例启动时间的情况下支持这些不同的目标，该应用程序在其最边缘处采用了一种专门的路由模式。

主要的入口点是 `cli.tsx`。该文件充当高速路由器。在加载任何复杂的应用逻辑、React 组件或 API 客户端之前，`cli.tsx` 会解析 `process.argv` 以识别请求的操作。它为特定的标志和子命令实现了“快速路径”(fast-paths)。例如，执行 `claude --version` 除了版本字符串本身之外，不需要导入任何模块，从而使其能够在几毫秒内打印并退出。这部分是通过利用构建时优化（例如 `bun:bundle` 指令）来执行基于特性开关的代码消除来实现的。

当检测到标准交互式命令时，`cli.tsx` 会推迟进入次要初始化阶段；但如果请求了替代的运行时目标，路由器则会完全走另一条分支。其中一个分支就是 MCP 服务器模式（`mcp.ts`）。当以此模式调用时，Claude Code 将作为一个独立的进程运行，严格通过标准输入和输出（STDIO）进行通信。该模式剥离了所有交互式 UI 组件，建立了一个专门为外部 MCP 客户端设计的隔离执行上下文。它维护着自己的内部 LRU 文件缓存（上限为 100 个条目），并将内部的 Zod 验证模式（schemas）自动转换为 MCP 规范所需的标准 JSON Schema 输出。

1.1.1　Schema 优先的 SDK 基础

入口点架构的很大一部分是专门为 Agent SDK 设计的。Claude Code 并没有为每种支持的语言手工编写类型定义，而是在 `entrypoints/sdk/` 中采用了 Schema 优先的设计。在这里，Zod 模式作为应用程序数据结构的无可争议的唯一可信数据源（source of truth）。

coreSchemas.ts 文件定义了所有可序列化的类型，包括配置、MCP 服务器设置以及 30 多种不同的生命周期钩子事件。这些 Zod 模式被输入到一个自动生成管道（scripts/generate-sdk-types.ts）中，由该管道生成并由 coreTypes.ts 导出 TypeScript 类型。

```
// Example of the SDK Control Protocol Schema definition pattern
export const SDKControlRequestSchema = z.discriminatedUnion("subtype", [
  InitializeRequestSchema,
  PermissionRequestSchema,
  McpMessageSchema
  // ...
]);
```

为了促进 Node.js CLI 进程与外部 SDK 实现之间的通信，controlSchemas.ts 定义了双向的进程间通信（IPC）协议。SDK 消费者通过标准输入发送 JSON 序列化的控制请求，CLI 在处理和响应之前会根据 SDKControlRequestSchema 对其进行验证。为了防止核心数据类型与控制协议类型之间出现循环依赖问题，该架构严重依赖于 lazySchema() 包装器，确保模式仅在验证期间明确需要时才会被求值。

此外，面向公众的 agentSdkTypes.ts 文件提供了一套存根函数（stub functions）。对于在 IDE 或外部脚本中集成 Claude Code 的开发者来说，这些强类型的存根记录了预期的 API 层面。实际的执行逻辑深深隐藏在宿主 CLI 中，但这一边界确保了类型安全、出色的 IDE 自动补全功能，并为 SDK 消费者提供了面向未来的保障。

1.2 CLI 路由与 I/O 传输

一旦 cli.tsx 确定用户正在调用标准的 CLI 命令，执行权就会移交给 cli/ 目录。这一层负责命令路由、I/O 编排和传输协商。它掌控着初始启动之后、核心查询执行引擎接管之前发生的一切。

1.2.1 延迟加载的命令处理器

Claude Code 中的命令被结构化为独立的处理器，位于 cli/handlers/ 目录中（例如 auth.ts、agents.ts、mcp.ts、plugins.ts）。这里一个关键的架构决策是使用动态导入。通过延迟加载这些处理器，在输入 claude help 或调用简单的实用命令时，就能避免因解析整个应用程序而产生的内存和 CPU 开销。

尽管它们是相互隔离的，但所有处理器都共享一致的执行模式。它们利用共享的服务层来进行分析、配置解析和错误格式化。此外，所有处理器都通过一个集中的退出实用程序（cli/exit.ts）终止。通过强制所有命令使用 cliOk() 或 cliError()，而不是将 process.exit() 调用散布在代码库各处，系统确保了一致的输出格式、可靠的遥测数据刷新以及更好的可测试性。

1.2.2 结构化与远程 I/O

CLI 操作的核心是 `print.ts`，它包含中央的读取-求值-输出循环（REPL）。这个循环完全与传输协议无关。它管理会话状态、工具执行和 MCP 服务器生命周期，而无需知道用户是在本地键盘上打字还是从云环境连接的。

为了将这个与传输无关的循环与外部世界连接起来，Claude Code 实现了两条不同的 I/O 路径：

- **结构化 I/O（`structuredIO.ts`）**：用于本地执行，通过 STDIO 进行通信。它读取并写入 NDJSON（换行符分隔的 JSON）控制消息。
- **远程 I/O（`remoteIO.ts`）**：在以 SDK 模式运行或连接到 Cloud Code Runtime (CCR) 时使用。它基于 HTTP/SSE 或 WebSockets 运行。

I/O 层中一个值得注意的工程细节是依赖于安全的 NDJSON 序列化器（`ndjsonSafeStringify.ts`）。虽然原始的 U+2028（行分隔符）和 U+2029（段落分隔符）字符在 JSON 字符串中是完全合法的，但当被 JavaScript 的字符串分割逻辑解析时，它们会充当实际的换行符。为了防止这些字符破坏 NDJSON 流并导致外部消费者崩溃，所有数据在跨越 I/O 边界之前，都要经过严格的转义函数处理。

1.2.3 权限竞速与传输多态性

在混合的本地/远程 AI 工具中，最复杂的用户体验挑战之一就是处理权限。Claude Code 在 `structuredIO.ts` 内部采用了“权限竞速”（Permission Racing）架构。当某个工具需要授权（例如，读取受保护的文件）时，系统会同时触发本地权限钩子（如根据配置文件自动确认特定目录的读取），并发送一个远程 `control_request` 请求人类批准。无论哪个先解决就赢得竞速，而失败者会立即被中止。这赋予了本地规则超低的延迟，同时也能优雅地降级为人工干预。

对于远程执行，Claude Code 在 `cli/transports/` 中实现了一个高弹性、运行时多态传输层。工厂函数 `getTransportForUrl()` 会检查环境变量和目标 URL 方案（scheme），以实例化以下三种传输之一：

1. **WebSocket 传输**：实时双向通信的默认方式。它具有用于重新连接的自动指数退避（exponential backoff）、10 分钟的重连预算（允许服务器优雅地终止空闲会话），以及防止主动代理超时的 ping/pong 保活机制。
2. **SSE 传输**：使用服务器发送事件（Server-Sent Events）进行长轮询读取，并使用批量 HTTP POST 进行写入。
3. **混合传输**：结合 WebSockets 用于低延迟读取，以及批量 HTTP POST 用于写入。

   架构说明：防止 Firestore 冲突
   SSE 和混合传输都严重依赖于 `SerialBatchEventUploader`。由于远程会话数据经常同步到云数据库（如 Firestore），向同一个文档发出的并发 POST 请求会相互覆

盖。上传器将所有传出事件强制放入一个严格的序列化队列中，将它们批量打包（每个 POST 最多 500 个项目）并执行背压（backpressure），从而保证数据的完整性。

1.3 全局状态初始化

虽然 CLI 路由器决定了执行权要转移到 *哪里*，但应用程序仍必须设置其运行的 *环境*。这分为两个领域：在 `init.ts` 中设置环境，以及在 `bootstrap/state.ts` 中进行内存状态管理。

1.3.1 预检初始化阶段

在需要完整的 CLI 模块（`cli/main.js`）之前，`cli.tsx` 会调用 `init()` 函数。这个预检（pre-flight）阶段对于建立应用程序的安全边界至关重要。它会验证用户配置文件、配置内部网络代理，并设置企业环境所需的 mTLS（双向 TLS）证书。

关键的是，`init.ts` 处理了重型依赖项的延迟加载。例如，初始化 OpenTelemetry 指标和追踪管道会占用大量的内存（约 400KB）。通过将此初始化推迟到主循环开始之前，应用程序保留了快速实用命令所需的超快启动时间。此外，`init.ts` 在启动序列中散布了大量 `profileCheckpoint()` 调用，使维护人员在调试启动缓慢问题时能够生成细粒度的启动性能火焰图。

1.3.2 中央 STATE 单例

传统的命令行应用程序通常在状态管理上苦苦挣扎，这往往导致深层嵌套的 Props 透传（prop-drilling）或混乱的全局变量。Claude Code 通过在 `bootstrap/state.ts` 中建立一个严格且类型安全的单例（singleton）解决了这个问题。

`State` 接口是运行时数据流的支柱。它包含超过 80 个不同的字段，跟踪从 API 成本指标和模型使用情况到活跃会话 UUID、插件注册表以及类似 cron 的后台任务等一切内容。当引导阶段调用 `getInitialState()` 时，它会读取当前工作目录（CWD），解析任何符号链接以建立规范的执行路径，并为会话生成一个唯一的 UUID。

为了防止意大利面条式的代码修改，`state.ts` 强制使用严格的访问器模式。模块不能随意覆盖状态字段；它们必须使用明确的、类型安全的 setter 函数。

```
// Example of the safe state mutation pattern
export function addModelUsage(model: string, tokens: number): void {
  const currentState = STATE.modelUsage.get(model) || 0;
  STATE.modelUsage.set(model, currentState + tokens);
  // Emit telemetry or trigger reactive updates as needed
}
```

`STATE` 单例也与应用程序的遥测优先（telemetry-first）设计紧密相连。在初始化期间，OpenTelemetry 的 `Meter`、`Logger` 和 `Tracer` 提供程序被直接附加到全局状态对象上。这确保了任何模块，无论它在工具执行或插件钩子中的嵌套层级有多深，都可以即时访问活跃的 Span 并发出结构化日志。

此外，状态系统还处理“粘性”会话行为。Beta 特性标志（例如 `afkMode` 或 `fastMode`）作为标头锁存器（header latches）实现。一旦通过命令行标志或提示词指令激活，状态单例会在该会话 UUID 的持续时间内将其锁定。这防止了不稳定行为的发生，比如提示词缓存可能在会话中途突然失效，仅仅是因为后续命令忘记传递正确的 CLI 标志。通过这一稳健的引导序列，Claude Code 确保在用户输入他们的第一条提示词时，一个具有弹性、类型安全且高度插桩检测的运行时环境已经准备好接收它。

第二章　核心数据结构与状态管理

2.1　中央类型系统与验证模式 (Schemas)

Claude Code 建立在严格、全面的 TypeScript 定义和运行时验证模式（Schema）的基础之上。由于该应用程序需要编排外部插件、复杂的本地 Hook 以及异步的语言模型交互，因此仅依赖编译时检查是远远不够的。该架构将其契约层分为编译时类型（存放在 `types/` 目录中）和运行时验证模式（在 `schemas/` 目录中管理）。

2.1.1　基于 Brand 模式的名义类型标识符

在分布式系统和复杂状态机中，隐蔽错误的最常见来源之一就是标识符混淆——例如在需要会话（Session）ID 的地方意外传入了用户（User）ID，因为两者通常都是字符串。Claude Code 完全通过使用 TypeScript 的"品牌类型（Branded Types）"模式实现名义类型（Nominal Typing）来彻底消除这一问题。

在 `types/ids.ts` 中定义，诸如 `SessionId` 和 `AgentId` 等标识符在编译时属于截然不同的类型：

```
export type SessionId = string & { readonly __brand: unique symbol };
export type AgentId = string & { readonly __brand: unique symbol };

export function toAgentId(id: string): AgentId {
  if (!/^a(?:.+-)?[0-9a-f]{16}$/.test(id)) {
    throw new Error(`Invalid Agent ID format: ${id}`);
  }
  return id as AgentId;
}
```

这种严格的类型控制贯穿于整个执行路径。当文本输入触发命令时，`AgentId` 会被附加到队列负载（Payload）中。当 Hook 回调触发或生成会话日志时，`SessionId` 会将这些分析数据关联起来。编译器强制保证 `AgentId` 绝不会意外地交叉污染需要 `SessionId` 的负载数据。

2.1.2 用于详尽匹配的辨识联合 (Discriminated Unions)

为了消除“字符串风格类型（stringly-typed）”的错误处理机制和脆弱的 switch 语句，系统高度依赖辨识联合（Discriminated Unions）。例如，插件系统定义了一个 `PluginError` 联合类型，包含 24 种不同的错误变体，每种变体都有特定的元数据。同样，Hook 事件系统使用包含 17 种 Hook 事件类型的辨识联合。

通过对共享的辨识字段（例如 `type`）进行 `switch` 判断，开发者可以充分利用 TypeScript 的详尽控制流分析。如果联合类型中新增了一个 Hook 类型或错误变体，编译器会立即标记出整个 monorepo 中遗漏的 `case` 语句，从而确保在不依赖运行时回退（Fallback）的情况下，对所有边界情况实现稳健的覆盖。

2.1.3 运行时验证与惰性 Schema 模式

虽然 TypeScript 处理了内部边界，但外部输入——如插件清单、模型上下文协议（MCP）服务器响应和配置文件——必须在运行时进行验证。Claude Code 使用 Zod 来保证这种结构完整性。

在大型 Zod 代码库中，一个主要的架构挑战是模块间的循环依赖：Schema 通常需要引用 TypeScript 接口，而这些接口反过来又使用 `z.infer<…>` 从其他 Schema 派生其类型。为了解决这个问题，Claude Code 采用了 *惰性 Schema*（*Lazy Schema*）模式。在 `schemas/hooks.ts` 中可以看到，Schema 被包裹在工厂函数或 Zod 的 `z.lazy()` 抽象中，以将计算推迟到该 Schema 被实际调用时进行。

```
export const buildHookSchemas = () => {
  return z.lazy(() => z.discriminatedUnion("type", [
    BashCommandHookSchema,
    PromptHookSchema,
    HttpHookSchema,
    AgentHookSchema
  ]));
};
```

2.1.4 规范的时间表示

对于数据分析、日志记录和远程同步而言，时间精度至关重要。由于浮点数的限制，JavaScript 原生的 `number` 类型在表示纪元纳秒（Epoch Nanoseconds）时会丢失精度。Claude Code 通过将所有时间数据标准化为 Google 的通用类型（Well-Known Type, WKT）`google.protobuf.Timestamp` 来解决这个问题。

在 `types/generated/google/timestamp.ts` 中，这段自动生成的 protobuf 代码使用了一个 64 位的 `Long` 抽象来表示秒（防止在 2262 年之后发生溢出），并使用一个单独的 32 位整数来表示

纳秒。代码库中的每个事件 Schema 都会委托该单一文件中的 `toJSON()` 和 `fromJSON()` 方法进行处理，从而建立一种不可破坏的、规范的时间传输格式（Wire Format）。

2.2 不可变配置与限制

一个稳健的 CLI 工具必须严格管理系统边界、API 限制和功能开关。在 Claude Code 中，`constants/` 目录作为这些边界的唯一、不可变的真相源（Source of Truth）。在设计上，该目录完全不包含任何依赖项。`constants/` 中的模块不会从应用程序的其余部分导入任何内容，从而消除了循环依赖的风险，并允许任何层（从最底层的解析器到最高层的 UI React 组件）安全地使用它们。

2.2.1 死代码消除与打包优化

为了保持 CLI 的响应速度并最小化占用空间，配置层针对构建时的摇树优化（Tree-shaking）进行了优化。文件大量使用 TypeScript 的 `as const` 断言，并导出扁平的字典或命名的原始值（Primitives）。

因为这些常量是可以静态分析的，打包工具可以在生产环境构建中积极地剔除未使用的数据。如果 `turnCompletionVerbs.ts` 中的某个特定的 UI 动词，或者 `xml.ts` 中的某个特定的 XML 标签在最终的依赖图中没有被引用，它会在编译期间直接被消除。

2.2.2 双层限制：客户端与 API

Claude Code 明确区分了 *客户端主动限制与 API 强制边界*。

`apiLimits.ts` 文件定义了 Anthropic API 的硬性约束（例如，每张图像最大 5MB，每个 PDF 最多 100 页，或每次请求最多 100 个媒体项）。这些值被预检验证逻辑用来实现快速失败，并在发起网络请求之前提供本地的、具有可操作性的错误信息。

相反，`toolLimits.ts` 定义了灵活的、客户端的应用程序约束。例如，工具结果字符串默认截断为 50,000 个字符，每个消息轮次的合计字符预算为 200,000 个字符。这使得系统的 `systemPrompts.ts` 缓存逻辑能够抢先截断冗长的本地工具输出（比如庞大的 `grep` 结果），以维持上下文窗口的稳定性，而不会触碰 API 的硬性限制。

2.2.3 功能门控与 Beta 注册表

实验性功能通过 `betas.ts` 和 `tools.ts` 进行集中管理。功能的可用性通过一个集中的注册表进行门控（Gated），而不是将布尔值检查散布在整个代码库中。

这些文件利用 `feature()` 工具函数（通过环境绑定提供）来确定是否应暴露实验性功能，如快速模式（Fast Mode）、高级上下文解析或多智能体协调器。这种模式控制着向 API 请求中注入

自定义 HTTP 请求头，并过滤传递给智能体的允许工具集，从而允许针对特定的用户群体进行安全的、渐进式的发布部署。

2.2.4 治理与安全隔离

至关重要的是，安全策略与操作逻辑是解耦的。`cyberRiskInstruction.ts` 文件包含了核心的防护指令——这些规则定义了可接受的安全协助（如代码审计）与被禁止活动之间的边界。通过将这些指令隔离到一个专用的常量文件中，该架构确保了治理规则的任何变更都需要安全团队进行明确的、局部的审查，且完全独立于智能体本身的执行机制。

2.3 响应式状态与副作用

尽管 Claude Code 是一个命令行界面，但它的终端渲染是由 Ink（一个基于 React 的框架）驱动的。因此，它需要一个稳健的响应式状态管理架构。`state/` 目录实现了一种分层方法，将持久化的外部环境（磁盘上的配置文件）与短暂的、高度交互的内部 UI 状态桥接起来。

2.3.1 框架无关的 Store 基础

位于状态层级结构最底层的是 `store.ts`。Claude Code 没有引入像 Redux 或 Zustand 这样沉重的依赖，而是实现了一个极简的、与框架无关的 Store。它暴露了一个简单的契约接口：`getState()`、`setState()` 和 `subscribe()`。

对该 Store 的更新是严格不可变的。应用程序状态定义 `AppState`，是通过 `AppStateStore.ts` 中的 `getDefaultAppState()` 工厂函数初始化的。这种解耦使得核心状态引擎能够被后台进程、异步网络处理器或测试套件查询和操作，而无需依赖 React 的渲染上下文。

2.3.2 基于 Selector 的 React 集成

为了将原生 Store 桥接到 React UI 中，`AppState.tsx` 提供了一个 `AppStateProvider` 和一系列自定义 Hook。主要的 Hook `useAppState(selector)` 在底层利用了 React 的 `useSyncExternalStore`。

这种设计针对渲染性能进行了深度优化。通过向该 Hook 传递一个纯净的 Selector 函数（定义在 `selectors.ts` 中），组件只订阅它们关心的精确状态切片。状态对比使用了严格的 `Object.is` 相等性校验。如果全局状态更新了，但派生出来的切片保持不变，组件就会避免重新渲染。同样地，`useSetAppState` 返回的是状态修改器的稳定引用，这意味着只分发（Dispatch）动作的组件永远不会发生意外的重新渲染。

2.3.3 面向高频流式传输的推测状态 (Speculation State)

语言模型流式传输引入了一个独特的状态管理挑战：高频更新（通常每秒几十个 Token）会导致严重的 UI 卡顿，如果在收到每个 Token 时都对消息数组进行不可变的展开（Spread）和重新分配。

为了解决这个问题，`AppState` 引入了“推测状态”——在状态树内部为快速变化的数据嵌入可变的 `{ current: …}` React refs。这使得流引擎能够即时将 Token 追加到由 ref 支持的缓冲区中，同时 React 定期批量处理视觉更新，从而将网络分块（Chunk）速率与终端帧率解耦。

2.3.4 副作用的单一收口点

状态层中最关键的架构决策或许就是如何处理副作用。在一个标准的 React 应用程序中，副作用（如保存到磁盘或追踪分析数据）通常散布在各个组件的 `useEffect` Hook 中。

Claude Code 将所有状态转换汇聚到一个集中的单一收口点：`onChangeAppState.ts`。该模块被注册为底层 Store `onChange` 事件的唯一订阅者。每当 UI 状态发生突变时——无论是通过键盘快捷键、斜杠命令还是对话框确认——都由这个单一的处理器来处理状态转换。

它负责：

- 将 UI 开关状态同步回持久化的外部设置 Store 中。
- 将权限模式的更改（例如从“ask”切换到“auto”）通知给底层 SDK 和模型上下文协议（MCP）服务器。
- 当上下文边界发生改变时，使内存缓存失效。
- 在活动的子智能体会话间传播环境变量的更新。

通过集中处理副作用，该架构保证了不管用户交互是如何触发的，视觉状态和底层操作能力（如工具权限）永远不会出现同步脱节的情况。

第三章　查询执行与任务编排

3.1　查询执行引擎

在 Claude Code 的核心是查询执行引擎，这是一个复杂的运行时环境，负责评估提示词、管理 LLM 交互以及执行操作约束。该引擎经过精心架构设计，旨在隔离副作用并保持可预测性，追求函数式编程的理想状态，即其核心步进机制充当一个纯归约函数（pure reducer function）：`(state, event, config)` $\rightarrow$ *state*

为了实现这一点，该架构高度依赖于依赖注入、不可变的配置快照以及响应式流原语。

3.1.1　依赖注入与不可变配置

查询执行周期的开始,是在可变的应用程序状态与执行上下文之间建立严格的边界。`query/config.ts` 模块会在每次查询调用时，精确地捕获一次当前会话状态、功能开关（feature gates）和环境限制的不可变快照。通过切断引擎对实时、可变存储的直接访问，Claude Code 确保了回合中途的更改（例如用户在另一个终端选项卡中修改设置）不会在单次模型评估期间导致竞态条件或逻辑不一致。

与这种数据不可变性相辅相成的是引擎的依赖注入容器（定义在 `query/deps.ts` 中）。执行引擎并没有硬编码导入外部服务——如模型客户端、文件系统访问器或 UUID 生成器——而是依赖于 `QueryDeps` 契约。

```
export interface QueryDeps {
  callModel: typeof callModelImpl;
  microcompact: typeof microcompactImpl;
  autocompact: typeof autocompactImpl;
  uuid: () => string;
  // ... other injected services
}
```

> 关于可测试性的说明：通过为 `QueryDeps` 接口使用 `typeof fn` 签名，测试替身（fakes）能够自动与生产环境的实现保持完美的同步。这消除了每个模块中脆弱的 `jest.spyOn()` 样板代码的需要，并确保核心执行引擎可以完全在内存中进行测试。

在生产环境中，工厂函数 productionDeps() 负责连接真实服务，而测试套件则传入隔离的模拟对象(mocks)。这种控制反转使得引擎能够安全地编排复杂的行为，例如上下文压缩(context compaction)，而不会与底层实现紧密耦合。

3.1.2 Token 预算与边际效益递减

查询引擎最关键的职责之一就是管理 token 预算。生成式 AI 模型偶尔会陷入递归循环，不断请求延续调用（continuation calls）却无法产生有意义的进展。query/tokenBudget.ts 模块通过双阈值评估系统来缓解这一问题。

首先，它监控绝对的 token 消耗量。如果操作超过了 COMPLETION_THRESHOLD（通常设置为分配的最大上下文窗口的 90%），引擎会强制停止执行，以保留剩余的 10% 用于关键的收尾任务和错误报告。

其次，且更重要的是，预算管理器实现了一种“边际效益递减（diminishing returns）”启发式算法。引擎会跟踪跨延续周期消耗的 token 增量。如果查询运行器发出 3 次或更多的延续请求，并且每轮的 token 增量降至 500 token 以下，引擎就会判定模型可能卡在了一个非生产性的循环中。

```
if (continuations >= 3 && deltaTokens < 500) {
  return {
    decision: 'stop',
    reason: 'diminishing_returns'
  };
}
```

这种启发式算法保证了 Claude Code 能够快速失败并保护用户的额度，而不是将 token 余量悄无声息地耗尽在无意义的文本生成上。

3.1.3 回合生命周期与停止钩子

当查询评估完成时——无论是成功完成还是因预算中断——引擎会启动由 query/stopHooks.ts 管控的清理阶段。停止钩子（stop hooks）并没有返回一个扁平的执行结果，而是被实现为一个异步生成器（async generator）。

在处理回合结束钩子时，该生成器会产出（yield）流事件、附件和进度消息。通过产出流而不是等待一个单一的庞大 Promise，CLI 能够在后台工作仍在进行时渲染实时的 UI 更新（例如，“正在提取记忆……”、“正在对任务进行分类……”）。

停止钩子编排了三个主要阶段：

1. 停止钩子（Stop Hooks）：清理例程，例如释放计算机使用权锁或保存会话状态。

2. 任务完成钩子 (Task Completed Hooks): 后处理操作,如记忆提取(自动整理/auto-dreaming)和遥测分类。

3. 队友空闲钩子 (Teammate Idle Hooks): 发送给系统中其他智能体的通知，指示当前线程已空闲，可以接收新任务。

至关重要的是,钩子引擎会检查 toolUseContext.agentId。如果当前执行是作为子智能体(subagent)运行，而不是面向用户的主线程，那么几乎所有的停止钩子都会被跳过。子智能体绝对不能触发 auto-dreaming 或释放全局文件锁，因为这样做会污染主线程的时间线并产生竞态条件。

3.2 后台任务与生命周期

因为 Claude Code 与开发者的文件系统和工具进行了深度集成，它经常需要管理长时间运行的后台进程。这些进程涵盖了从本地 shell 命令（例如编译二进制文件或运行开发服务器）到基于云的远程执行会话以及自主运行的后台子智能体。

3.2.1 任务管理架构

为了防止这些长时间运行的任务淹没主要的交互式提示符，Claude Code 采用了统一的后台任务注册表。该注册表充当一个集中的进程管理器，跟踪所有分离（detached）操作的生命周期、标准输出流和退出代码。

当一个智能体执行带有 background: true 参数的 bash 命令时，或者当生成一个工作智能体（worker agent）来处理繁重的重构任务时，执行引擎会将该进程注册到此任务管理器中。随后，主查询引擎可以继续处理用户的下一个提示词而不会被阻塞。这些任务的生命周期受到全面监控；如果用户退出 Claude Code，任务管理器将负责优雅地终止任何孤儿进程或远程云会话。

3.2.2 延迟加载的命令实现

为了向用户展示这些后台进程,Claude Code 提供了一个专用的 CLI 命令,位于 commands/tasks/ 目录中。该实现突显了整个应用程序中使用的一个关键架构模式：为提升 CLI 性能的延迟加载（lazy loading）。

该命令被拆分为两个文件：

- tasks.ts: 定义了轻量级的元数据（名称、诸如 bashes 等别名、描述）和一个 load() 函数。
- tasks.tsx: 包含了繁重的 React/Ink 组件逻辑，用于渲染交互式的任务管理对话框。

```
// tasks.ts (Command Registry)
export default {
```

```
  name: 'tasks',
  aliases: ['bashes'],
  type: 'local-jsx',
  load: async () => (await import('./tasks.tsx')).default
} satisfies Command;
```

通过使用动态导入实现 LocalJSXCommand 接口，Claude Code 阻止了 Node.js V8 引擎在初始启动期间解析和编译 React、Ink 以及复杂的 UI 组件。只有当用户显式输入 /tasks 时，才会计算并执行繁重的 tasks.tsx 文件，从而使应用程序启动的关键路径保持极快的速度。一旦被调用，tasks.tsx 就会接管控制权，渲染一个动态的终端 UI，该 UI 订阅了后台任务注册表，允许用户监控或终止活跃的作业。

3.3 多智能体协调器模式

对于涉及多个不同文件、独立的研究阶段和测试的高度复杂的请求，单一的顺序型 LLM 执行往往力不从心。为了解决这个问题,Claude Code 实现了一个由 coordinator/coordinatorMode.ts 模块驱动的强大的多智能体编排系统。

当该系统启用时,它从根本上将架构从单一的交互式循环,转变为一个项目经理(协调器/Coordinator)指挥多个并行工作的“工人”智能体（Worker agents)。

3.3.1 编排系统设计

协调器智能体作为面向用户的接口。其主要的系统提示词被增强，以强调并行任务的执行。协调器的任务不是直接编写代码,而是分析用户的请求,将其分解为独立的子任务,并使用 TeamCreate 和 SendMessage 内部工具来启动工作智能体。

例如，如果被要求“迁移数据库模式并更新前端组件”，协调器可以并发地生成一个 agent_db 智能体和一个 agent_frontend 智能体。协调器等待它们的结果，在向用户展示最终结果之前，对输出进行综合并验证更改。这种并发性极大地减少了大规模重构所需的实际耗时(wall-clock time)。

3.3.2 能力模式与工具隔离

并非所有任务都需要用到 Claude Code 的全部能力。协调器系统支持根据 CLAUDE_CODE_SIMPLE 环境变量将工作智能体配置为不同的能力模式。

- 简单模式工作者 (Simple Mode Workers): 被限制为只能使用基础的 Bash、Read 和 Edit 工具。对于不需要复杂推理或外部 API 调用的直接实现任务，它们非常高效。

- 全能模式工作者 (Full Mode Workers): 配备了所有标准工具、MCP (模型上下文协议) 能力以及自定义技能 (skills)。这些主要用于开放式研究或调试任务。

至关重要的是,该架构必须防止工作智能体递归地生成无限层的子智能体。coordinatorMode.ts 模块通过 INTERNAL_WORKER_TOOLS 限制集合来强制执行此操作:

```
const INTERNAL_WORKER_TOOLS = new Set([
  'TeamCreate',
  'TeamDelete',
  'SendMessage',
  'SyntheticOutput'
]);
```

当构建工作智能体的上下文时, 执行引擎会显式地从其能力列表中剥离这些工具。只有顶层协调器有权限管理团队的拓扑结构。

3.3.3 会话恢复与功能开关

因为多智能体操作与标准执行模式有着显著的差异,协调器模式受到了严格的功能开关管控。它需要环境变量选择加入(CLAUDE_CODE_COORDINATOR_MODE)和服务器端的 Statsig 标志(tengu_scratch)双重条件才能激活。

这种双层开关管控为会话持久化带来了独特的挑战。如果用户启动了一个多智能体会话,将其挂起, 然后稍后在不同的终端环境中恢复它, 系统必须同步协调器的状态。matchSessionMode() 函数通过检查加载的会话数据来处理这个问题。如果它检测到被恢复的会话最初是在协调器模式下运行的,它会将相应的环境变量动态注入回运行时进程中,从而确保编排拓扑在应用程序重启后保持完好无损。遥测钩子会自动记录这些转换(例如,tengu_coordinator_mode_switched),以便跟踪在不同环境下的功能采用率和稳定性。

第四章　工具与技能生态系统

4.1　核心工具框架

Claude Code 通过一个健壮的异步工具框架与外部世界（用户的文件系统、操作系统命令和网络资源）进行交互。该框架主要位于 services/tools/ 目录中，其设计围绕流式执行、精确的并发控制和广泛的遥测技术展开。

4.1.1　工具编排与划分

当 LLM 决定使用工具时，它会在消息中发出一个或多个 ToolUseBlock 元素。处理这些请求的入口点是在 toolOrchestration.ts 中定义的编排器（orchestrator）。

编排器的主要职责是将工具调用划分为批次，并管理整个执行管道。由于 LLM 以流式传输其响应，工具块会增量到达。编排器必须决定工具何时被完全解析并准备好运行，从而确保工具在其输入可用时立即执行，而不是等待整个 LLM 响应完成。

4.1.2　流式工具执行器

执行引擎的核心是 StreamingToolExecutor。该类接收排队中的工具调用，并基于复杂的并发模型准确决定它们应该如何以及何时运行。

Claude Code 中的每个工具都通过 isConcurrencySafe 标志显式声明其并发安全性：

- 安全工具（Safe Tools）：诸如 FileReadTool 或 GrepTool 之类的工具被标记为并发安全。执行器将并行运行这些工具，直到达到定义的上限（由 CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY 控制，默认为 10）。
- 独占工具（Exclusive Tools）：改变状态的工具（如 BashTool 或 FileWriteTool）在并发运行时存在根本性的不安全因素。执行器保证这些工具串行运行，在它们完成之前会阻塞其他工具。

这种选择性加入（opt-in）的并发机制防止了竞争条件（例如两个工具试图同时写入同一个文件），同时最大化了读密集型操作的 I/O 吞吐量。

StreamingToolExecutor 还负责多层中止传播（abort propagation）策略。如果用户中断进程，取消操作会通过父级 AbortController 进行传递。如果像 BashTool 这样的关键工具发生灾难性故障，专用的子控制器会将中止操作级联到同一批次中的同级工具，从而防止依赖先前状态的工具进行不必要的执行。

```
export class StreamingToolExecutor {
  // 如果并发规则允许，
  // 将工具添加到队列并触发执行
  public addTool(toolBlock: ToolUseBlock): void {
    const tool = this.findToolByName(toolBlock.name);
    this.queue.push({ toolBlock, tool });
    this.scheduleNext();
  }

  // 执行 isConcurrencySafe 规则的内部调度器
  private scheduleNext(): void {
    // 评估当前运行的工具与排队的工具...
  }
}
```

4.1.3 单一工具执行与钩子（Hooks）

当一个工具被批准运行时，控制权会转移到 toolExecution.ts，它负责管理单一工具调用的生命周期。这包括解析用户权限、运行拦截器钩子、执行工具逻辑以及记录详细的 OpenTelemetry (OTel) 跨度（spans）。

执行生命周期在很大程度上利用了在 toolHooks.ts 中定义的钩子系统：

1. PreToolUse 钩子：在工具执行之前触发。这些钩子可以检查输入、根据配置规则自动授予权限、修改参数或完全阻止执行。这是 Claude Code 强制执行安全边界的地方。

2. PostToolUse 钩子：在工具成功完成后触发。这些钩子格式化输出、向用户界面发送文件附件，或调整有效载荷（payload）以适应特殊的外部集成。

3. PostToolUseFailure 钩子：如果工具抛出错误则被调用，允许进行优雅降级、提供自定义错误消息或收集遥测数据。

   设计说明：钩子架构将核心执行循环与特定领域的规则解耦。例如，决定特定 bash 命令是否需要用户确认的逻辑是作为 PreToolUse 钩子实现的，而不是硬编码在 BashTool 本身之中。

执行和钩子的结果作为 MessageUpdate 对象返回给编排器。StreamingToolExecutor 缓冲已完成的结果，以确保它们按照请求的确切顺序发送回 LLM，从而保持对话上下文的确定性。

4.2 技能管理与发现

除了基础工具之外，Claude Code 还引入了“技能（Skills）”——这是一种更高维度的抽象，它将特定领域的专业知识、命令和工具捆绑成具有凝聚力的包。技能生态系统将内置功能与动态发现的扩展统一了起来。

4.2.1 技能命令接口

用于与技能交互的用户界面在 commands/skills/ 目录中实现。通过使用 CLI 的标准命令模式，/skills 命令会为用户触发一个交互式菜单。

为了最小化初始应用程序的有效载荷，技能 UI 依赖于一种延迟加载（lazy-loaded）的基于组件的架构。index.ts 文件仅定义命令元数据和一个动态导入路径：

```
export const skillsCommand = {
  name: 'skills',
  description: 'Manage Claude Code skills', // 管理 Claude Code 技能
  load: () => import('./skills.tsx')
};
```

当被调用时,命令上下文会被传递给 skills.tsx 中的 call() 函数,该函数渲染一个 SkillsMenu React 组件。该菜单会基于当前发现的技能动态填充，并连接一个 onDone 回调以处理优雅退出，当用户关闭界面时通知全局命令系统。

4.2.2 模型上下文协议 (MCP) 集成

技能生态系统的一个关键方面是它对模型上下文协议（Model Context Protocol, MCP）的一等（first-class）支持。MCP 为 AI 模型提供了一种跨不同环境与外部工具、API 和文件系统交互的标准化方式。

在工具框架内，MCP 工具通过标准前缀（例如 mcp__）进行动态识别。isMcpTool() 实用函数决定给定的工具块是否应该路由到活动的 MCP 服务器，而不是内部的 Node.js 执行器。

当执行 MCP 工具时，标准工具钩子会被适配以处理 MCP 特有的语义：

- 身份验证处理：解析错误以查找特定签名（如 McpAuthError），从而允许 CLI 拦截失败并在自动重试工具执行之前提示用户输入凭据。
- 输出格式化：后置钩子处理标准化的 MCP 响应格式（如嵌入图像或多部分文本），并将它们转换为 Ink 渲染引擎可以显示的 Claude Code 原生终端附件。

这种无缝集成允许开发者构建本地或远程的 MCP 服务器，Claude Code 可以自动发现并使用它们，就像它们是原生编译的工具一样。

4.3 插件系统脚手架

虽然技能代表了特定领域的能力（通常在后台透明运行），但 Claude Code 也通过其插件系统（Plugin System）为面向用户、可切换的扩展奠定了基础。该架构位于 plugins/ 目录中，旨在为用户提供对可选 CLI 功能的细粒度控制。

4.3.1 内置插件与用户插件

插件目录在结构上进行了划分，以处理不同类别的可扩展性。plugins/bundled/ 目录目前作为架构脚手架，通过 index.ts 为内置插件提供入口点：

```
import { registerBuiltinPlugin } from '../builtinPlugins.js';

export function initBuiltinPlugins() {
  // 为未来的内置插件提供脚手架
}
```

理解捆绑的插件（plugins）和捆绑的技能（skills）（位于 src/skills/bundled/ 中）之间的架构区别至关重要：

- 捆绑技能（Bundled Skills）：这些是具有复杂设置逻辑的专门功能，当检测到其目标环境时会自动启用（例如 claude-in-chrome 浏览器自动化技能）。
- 捆绑插件（Bundled Plugins）：这些是随 CLI 一起提供的可选功能，但显式受用户控制。它们被设计为自动出现在 /plugin 用户界面中，提供了高可发现性，而不会强制用户采用。

4.3.2 未来的可扩展性

plugins/bundled/ 脚手架代表了一条战略性的迁移路径。随着 CLI 的演进，目前硬编码或作为自动启用技能实现的功能，将过渡到这种标准化的插件模型中。

通过在 CLI 启动的早期阶段执行 initBuiltinPlugins()（如第一章所述），系统将这些功能注册到一个统一的全局注册表中。该目录刻意不包含外部 npm 依赖，以确保核心 CLI 包保持轻量级，同时为未来由社区编写的插件生态系统建立一个健壮的契约。

> 设计说明：通过使用完全相同的 registerBuiltinPlugin() 接口来管理内置功能和未来的外部插件，该架构保证了第一方功能不依赖于隐藏的 API。这为希望扩展 Claude Code 功能的第三方开发者强制执行了一个干净、稳定的契约。

结论

本书对 Claude Code 的架构与实现进行了全面的探讨。从权限系统和工具基础架构，到流式 I/O 处理与命令行（CLI）界面，我们从系统层面剖析了现代 AI 驱动的编码助手是如何构建的。

其模块化的设计、对 TypeScript 类型系统的广泛应用，以及通过权限管理对安全性的高度关注，充分展示了如何将软件工程的最佳实践应用于 AI 辅助开发这一充满挑战的领域。